DeepLearning

Compte rendu Projet MAYDAY/URGENCY Detector

COLAS Maxime, DOS SANTOS Duncan et FERRI Corentin

7 mars 2026

Table des matières

1 Introduction **1**

2 Choix méthodologiques et organisation du projet **1**

3 Pipeline du projet **1**

4 Création du dataset **2**
4.1 Définition des classes 2
4.2 Méthode de génération 2
4.3 Répartition des données 3

5 Expérimentations et résultats **3**

6 Analyse critique **4**
6.1 Difficultés 4
6.2 Limites 5
6.3 Perspectives d'amélioration 5

7 Conclusion **6**

Table des figures

1 Répartitions du dataset entre enregistrement naturel et synthétique 3
2 Performances selon la taille du dataset 3

1 Introduction

Dans le domaine aéronautique, la communication radio entre les pilotes et les tours de contrôle est cruciale pour la sécurité des vols. Une interprétation rapide et précise des messages peut faire la différence dans la gestion des situations d'urgence.

L'objectif de ce projet est de développer un programme capable de détecter automatiquement le niveau d'alerte présent dans ces messages radio. Pour cela, nous avons choisi d'utiliser un modèle Transformer, entraîné sur un dataset spécialement créé afin de représenter les différents niveaux d'urgence rencontrés dans les communications aéronautiques.

2 Choix méthodologiques et organisation du projet

Pour permettre un travail simultané sur le projet, nous avons choisi d'utiliser Google Colab avec un notebook partagé. Cependant, lors de l'importation des fichiers audio, nous avons rencontré un problème : les fichiers n'étaient accessibles qu'en local. Pour contourner cette limitation, nous avons d'abord créé un dépôt GitHub afin d'y centraliser nos fichiers via la commande `git pull`. Toutefois, en raison de la taille importante des fichiers audio, cette solution s'est rapidement révélée insuffisante. Nous avons alors opté pour Google Drive, qui offre un espace de stockage plus conséquent et facilite la gestion et l'accès aux données pour tous les membres de l'équipe.

3 Pipeline du projet

Le pipeline de notre projet a été conçu pour transformer des fichiers audio bruts en prédictions fiables de niveaux d'alerte. Les fichiers audio sont d'abord centralisés sur Google Drive afin de permettre un accès partagé et simplifié pour tous les membres de l'équipe. Chaque fichier est associé à un label indiquant le niveau d'alerte : normal, urgent (PAN PAN) ou détresse (MAYDAY), ce qui facilite l'organisation et la préparation des données pour l'entraînement.

Avant toute analyse, les fichiers audio sont transcrits en texte à l'aide du modèle Whisper. Cette étape permet de convertir le contenu audio en un format exploitable par un modèle Transformer. Le choix de Whisper a été motivé par sa capacité à produire des transcriptions précises même avec un nombre limité d'échantillons, ainsi que par sa possibilité de transcrire dans plusieurs langues.

Les transcriptions sont ensuite prétraitées et converties en vecteurs numériques grâce aux tokenizers adaptés à RoBERTa. Cette étape standardise les données et prépare le modèle à l'apprentissage supervisé.

Le modèle RoBERTa, pré-entraîné sur de larges corpus textuels, est fine-tuné sur notre dataset spécifique pour classer correctement les transcriptions selon les trois niveaux d'urgence. Ce choix permet d'exploiter la puissance des modèles de langage pour la compréhension sémantique des messages radio, plutôt que d'utiliser directement des modèles audio. De plus, grâce au pré-entraînement du modèle, son utilisation sur un dataset personnalisé comme le nôtre est plus rapide et efficace.

Durant l'entraînement, le modèle est évalué sur un jeu de données que nous avons créé avec nos propres voix, incluant des erreurs de prononciation. Des voix générées par IA ont éga-

lement été utilisées afin d'introduire différents tons, vitesses et accents. À l'issue de ces tests, nous récupérons les performances du modèle, telles que la loss, l'accuracy, et d'autres métriques pertinentes.

Enfin, le pipeline permet de traiter de nouveaux messages audio en direct via un enregistrement micro. Ces fichiers sont transcrits, tokenisés et classifiés immédiatement, offrant ainsi une utilisation proche du temps réel.

4 Création du dataset

4.1 Définition des classes

La première étape de notre projet a consisté à classifier les différents types de messages en trois niveaux d'urgence distincts.

Le premier niveau d'urgence, intitulé *NORMAL*, correspond aux messages de routine et regroupe l'ensemble des communications radio standards. Par exemple, il peut s'agir d'un message radio indiquant si un avion est autorisé ou non à décoller d'une piste donnée.

Le deuxième niveau d'urgence, appelé *URGENCY*, concerne les situations pouvant affecter la sécurité de l'avion sans pour autant nécessiter une assistance immédiate. Ce type de message peut notamment contenir le terme « PAN PAN » et peut survenir, par exemple, dans le cas d'un problème médical à bord impliquant un passager.

Le dernier niveau d'urgence, nommé *DISTRESS*, correspond aux situations dans lesquelles l'avion est confronté à un danger grave et imminent nécessitant une assistance immédiate. Ce type de message peut contenir le terme « MAYDAY » et peut se produire, par exemple, lors d'une panne totale des moteurs de l'avion.

4.2 Méthode de génération

Afin de garantir une meilleure robustesse de notre modèle vis-à-vis des conditions réelles d'utilisation, nous avons choisi de contribuer nous-mêmes à la génération de données. En effet, le dataset initial est composé d'une deux centaine d'enregistrements audio originaux répartis entre trois locuteurs différents. Cette diversité permet d'enrichir notre jeu de données avec des voix, des intonations et des accents variés.

De plus, nous avons adapté notre manière de parler en fonction du niveau de gravité des messages. Par exemple, pour un message de type *DISTRESS*, le locuteur adopte généralement un débit de parole plus élevé que pour un message de type *NORMAL*. Nous avons également introduit volontairement certaines hésitations afin de reproduire au mieux l'authenticité des communications radio.

Enfin, afin d'augmenter la taille du dataset et de pallier le faible nombre d'enregistrements disponibles, nous avons généré synthétiquement 800 messages audio à l'aide d'outils d'intelligence artificielle. Cette approche permet non seulement d'élargir la couverture des messages possibles, mais également de simuler une plus grande diversité de conditions réelles, notamment par l'ajout de bruit de fond, de voix différente ainsi que de vitesse de parole.

Source	Classes	Nombre d'audios
Enregistrement naturel	NORMAL/URGENCY/DISTRESS	≈ 200
Enregistrement synthétique	NORMAL/URGENCY/DISTRESS	≈ 800

FIGURE 1 – Répartitions du dataset entre enregistrement naturel et synthétique

4.3 Répartition des données

Notre projet comporte ainsi environ 1000 fichiers audio. La répartition de ces données entre les différentes catégories est présentée dans le tableau ci-dessous :

5 Expérimentations et résultats

Pour évaluer la performance de notre modèle, nous avons réalisé plusieurs expérimentations en augmentant progressivement la taille du dataset. La première expérience a été menée avec un petit nombre d'échantillons (10 fichiers audio) afin de valider la faisabilité du pipeline et d'identifier les principales difficultés liées à la transcription et à la classification. Ce jeu de données était constitué uniquement de nos propres voix, incluant des erreurs de prononciation et des variations d'accent, ce qui a permis de détecter les limites initiales du modèle.

Ensuite, nous avons élargi le dataset à 500 fichiers audio, avec des fichiers audio créer par IA permettant plus de variation de voix et d'accent. Cela a permis d'améliorer significativement la performance du modèle, la détection des niveaux d'urgence est devenue plus précise.

Enfin, avec un dataset complet de 1000 fichiers audio, le modèle a atteint des performances optimales. Les tests ont montré que le transformer pouvait correctement identifier les niveaux d'alerte sur des fichiers audio variés, incluant des voix générées par IA, de différentes tonalité, robotisé ainsi que accélérer ou bruité. Ces résultats confirment la robustesse et la capacité de généralisation du modèle.

Cette progression permet de comparer les résultats en fonction de la taille du dataset et de dresser un tableau synthétique des performances obtenues à chaque étape, incluant également des tests sur un fichier audio de type "urgency" généré par IA ainsi qu'un fichier de type "distress" enregistré avec nos propres voix.

Dataset	Loss	Accuracy	Test 1	Confiance	Test 2	Confiance
10 audios	1.111254	0.5	Urgency	$\begin{pmatrix} normal & 0.3366 \\ urgency & 0.3725 \\ distress & 0.2910 \end{pmatrix}$	Urgency	$\begin{pmatrix} normal & 0.3371 \\ urgency & 0.3641 \\ distress & 0.2987 \end{pmatrix}$
100 audios	1.072131	0.7	Distress	$\begin{pmatrix} normal & 0.3345 \\ urgency & 0.3183 \\ distress & 0.3471 \end{pmatrix}$	Distress	$\begin{pmatrix} normal & 0.3373 \\ urgency & 0.3210 \\ distress & 0.3417 \end{pmatrix}$
1000 audios	0.192543	0.94444	Urgency	$\begin{pmatrix} normal & 0.0022 \\ urgency & 0.9648 \\ distress & 0.0330 \end{pmatrix}$	Distress	$\begin{pmatrix} normal & 0.0093 \\ urgency & 0.3607 \\ distress & 0.6299 \end{pmatrix}$

FIGURE 2 – Performances selon la taille du dataset

Le tableau 2 présente les performances de notre modèle selon la taille du dataset utilisé pour l'entraînement. Les colonnes indiquent : la taille du dataset, la valeur de la loss, l'accu-

racy obtenue sur le jeu de test, ainsi que les prédictions et les probabilités associées pour deux exemples distincts.

On observe clairement une amélioration progressive du modèle lorsque la taille du dataset augmente :

— Avec un petit dataset de 10 fichiers audio, le modèle a une précision très limitée (accuracy = 0,5). Les prédictions sont incertaines, comme le montre la distribution des probabilités proche de l'égalité entre les classes. Ce résultat reflète le manque de diversité et le faible nombre d'exemples pour apprendre correctement les niveaux d'urgence.

— Avec 100 fichiers audio, le modèle commence à mieux distinguer les classes (accuracy = 0,7). La loss diminue légèrement et les probabilités associées aux classes deviennent plus distinctes, montrant une meilleure confiance dans les prédictions. Cela illustre l'importance d'augmenter le nombre d'exemples pour affiner l'apprentissage.

— Avec 1000 fichiers audio, le modèle atteint des performances optimales (accuracy $\approx$ 0,944) et la loss est fortement réduite. Les probabilités pour chaque classe montrent une confiance beaucoup plus élevée dans les prédictions correctes, même pour des exemples variés. Cependant, certains fichiers restent difficiles, notamment lorsque la phrase est piégeuse ou ambiguë, comme le montre la distribution des probabilités pour le second test de 1000 fichiers.

Ces résultats confirment que la taille et la diversité du dataset sont des facteurs déterminants pour la robustesse et la précision du modèle. L'évolution de la loss et de l'accuracy montre que l'ajout d'exemples supplémentaires permet d'améliorer considérablement la capacité du modèle à généraliser sur de nouveaux fichiers audio.

6 Analyse critique

6.1 Difficultés

La première difficulté rencontrée concernait la possibilité de travailler simultanément sur le code et le dataset. Pour résoudre ce problème, nous avons utilisé Google Drive, permettant à toute l'équipe d'accéder aux mêmes données en temps réel et de collaborer efficacement.

Une autre difficulté liée au dataset était la taille des fichiers. Avec près d'une centaine de fichiers initiaux, il a été nécessaire de trouver un espace de stockage en ligne suffisant pour contenir l'ensemble des fichiers. Google Workspace étant payant, nous avons choisi Google Drive, qui représentait une alternative simple et efficace.

La création du dataset a constitué un troisième défi. Cette étape, cruciale, était longue, répétitive et fastidieuse. Pour nous faciliter la tâche, nous avons enregistré une centaine de fichiers avec nos propres voix, puis utilisé une IA pour générer les fichiers audio restants en lui fournissant le texte correspondant.

Enfin, l'enregistrement en direct des phrases a posé problème, car Python n'a pas d'accès natif au micro de l'ordinateur. Il a donc été nécessaire de passer par Google Colab pour réaliser ces enregistrements audio en direct.

6.2 Limites

Malgré ses performances, notre modèle présente plusieurs limites importantes :

— **Limite linguistique :** le modèle ne comprend que les phrases en anglais. Toute saisie dans une autre langue risque de ne pas être interprétée correctement.

— **Préparation et entraînement longs :** chaque fichier audio doit être transcrit avant l'entraînement, ce qui allonge considérablement le temps de préparation du dataset.

— **Robustesse limitée :** le modèle n'a pas été entraîné sur des fichiers audio bruités, avec des accents très marqués, des intonations variées ou des voix de synthèse variées, ce qui limite sa capacité à généraliser.

— **Taille du dataset :** même avec 1000 fichiers audio, le dataset reste relativement petit pour un modèle Transformer, ce qui peut limiter sa capacité à apprendre toutes les variations possibles de messages radio.

— **Dépendance à la qualité de la transcription :** la performance finale dépend fortement de la précision de Whisper. Les erreurs de transcription peuvent entraîner des classifications incorrectes.

— **Déploiement en temps réel :** l'enregistrement et la transcription en direct sont contraints par la latence de Colab et l'accès au micro via le navigateur.

6.3 Perspectives d'amélioration

Plusieurs pistes pourraient améliorer la performance et la portée du projet :

— **Support multilingue :** étendre le dataset à d'autres langues et ajouter une fonction de détection automatique de la langue après transcription, afin de rendre le modèle universel.

— **Augmentation du dataset :** enregistrer davantage de fichiers, inclure des voix variées, des intonations différentes et des environnements bruyants pour améliorer la robustesse.

— **Prétraitement du bruit :** intégrer des méthodes de réduction de bruit et d'amélioration audio avant la transcription pour diminuer les erreurs.

— **Optimisation temps réel :** explorer des solutions pour réduire la latence de l'enregistrement et de la classification en direct.

7 Conclusion

Ce projet a permis de développer un système capable de classifier automatiquement des fichiers audio selon leur niveau d'alerte, en combinant la transcription automatique via Whisper et la classification avec un modèle Transformer basé sur RoBERTa.

Nous avons pu observer que la performance du modèle s'améliore significativement avec l'augmentation de la taille du dataset, passant d'une précision médiocre sur un petit nombre de fichiers à une accuracy élevée ($\approx$ 94,4%) sur un dataset complet de 1000 fichiers audio. L'utilisation de fichiers générés par IA et de voix variées a également renforcé la robustesse et la capacité de généralisation du modèle.

Malgré ces résultats encourageants, le système présente des limites, notamment sa dépendance à la langue anglaise, l'absence de bruit ambiant dans les enregistrements, et la difficulté à gérer des phrases très ambiguës ou piégeuses. Ces observations ouvrent la voie à des améliorations futures, telles que l'intégration de la reconnaissance multilingue, l'ajout de fichiers bruités, ou l'enrichissement du dataset avec des accents et tonalités variés.

En résumé, ce projet démontre la faisabilité et l'efficacité d'un pipeline audio→texte→classification pour des messages radio d'aviation, tout en identifiant clairement les axes d'amélioration pour rendre le système plus universel et robuste.